

ADNEKO

AUTONOMOUS COMPANY

ESTADO REAL DEL DESARROLLO

Qué se ha construido · Qué falta ·

Documento	AC-STATUS-001 · v1.0
Fecha	22 septiembre 2026
Snapshot técnico observado	287d68bf624f89a9ec8a645a56aa0dc3e31151b8
PR de trabajo	PR #18 · OPEN · MERGEABLE · NOT MERGED
Estado de release	NOT CERTIFIED

Documento de claridad ejecutiva para fundador, equipo técnico e inversionistas. No sustituye la certificación técnica del release.

Autonomous Company NO es hoy una aplicación terminada que el usuario pueda abrir y operar de punta a punta. Sí existe un backend avanzado y material: persistencia PostgreSQL, runtime de proyectos, misiones, recuperación, seguridad, gobierno de capacidades, incidentes, efectos externos, evidencias, pruebas y herramientas de demo/certificación. Lo que falta es convertir ese motor en el cockpit original de producto - MAPA + DASHBOARD + CHAT + BRAIN, y después cerrar VOICE - y demostrar un flujo completo visible, persistente y repetible.

“ADNEKO Autonomous Company está en fase de integración de producto. El motor backend y la infraestructura de autonomía gobernada están implementados y bajo hardening; el cockpit frontend original aún no está terminado y el release actual no está certificado. El siguiente hito es una demo E2E visible donde una operación iniciada desde el dashboard atravesase el runtime, autoridad, persistencia y evidencia y vuelva al usuario con resultado verificable.”

SUPERFICIE	FUNCIÓN
MAP	Grafo vivo de empresa: Company → Department → Function → Capability → Agent/Workflow → Execution.
DASHBOARD	Prioridades, alertas, aprobaciones, misiones, ejecuciones, coste/riesgo/resultado y estado operativo.
CHAT IA	Interfaz de operación por lenguaje natural, aterrizada en Company Brain, estado y capacidades gobernadas.
VOICE	Comando, consulta y operación por voz usando la misma autoridad; no un sistema paralelo.
BRAIN	Contexto institucional: productos, servicios, precios, clientes, proyectos, políticas, contratos, decisiones y memoria operativa.

Referencia visual original / objetivo de interfaz (NO es evidencia de runtime ejecutado):



Referencia de diseño original: mapa empresarial con ADNEKO Brain central y dominios operativos alrededor.

Qué se ha construido

El trabajo comprobado en el repositorio se concentra en el motor de ejecución, continuidad, gobierno y evidencia. A continuación se separa lo implementado de lo visible para el usuario.

ÁREA / COMPONENTE	ESTADO HOY	QUÉ SIGNIFICA
Arranque de aplicación / servidor HTTP	IMPLEMENTADO	Existe bootstrap de producción, servidor API, configuración y ciclo start/stop.
PostgreSQL y migraciones	IMPLEMENTADO	Catálogo actual declara EXPECTED_SCHEMA_VERSION = 71, con 71 versiones lógicas de esquema.
Runtime de proyecto	IMPLEMENTADO	API y servicios para ejecutar operaciones de proyecto con persistencia e idempotencia.
Estado / continuidad de proyecto	IMPLEMENTADO	Versionado de estado, checkpoints, commitments, decisiones, conocimiento y continuidad.
Mission Engine	IMPLEMENTADO	Políticas, ledger, locks, attempts, wakeups, resume, recovery y escalación humana.
Autonomous Project Loop	IMPLEMENTADO	Ciclos autónomos con reglas de continuación, paro, autoridad y manejo de escalación.
Effect / Execution Ledger	IMPLEMENTADO	Registro durable de efectos, idempotencia, lifecycle, verificación, recovery y reconciliación.
Recovery	IMPLEMENTADO	Recuperación de ejecuciones abandonadas, efectos inciertos y misiones interrumpidas.
JWT / seguridad	IMPLEMENTADO	Proveedor JWT HS256, scopes, issuer/audience y control de autoridad.
Incidentes operativos	IMPLEMENTADO	Listado, detalle, claim, acknowledge, repair, resolve y post-resolution.
Gobierno de capacidades	IMPLEMENTADO	Propose/approve/reject/cancel/apply de capacidades de producción.
Business artifacts	IMPLEMENTADO	Superficie API para artifacts internos, comandos e historial.
Health / readiness	IMPLEMENTADO	Comprobaciones de aplicación, DB, schema, IA y runtime.
Demo técnica automatizada	EXISTE EN CÓDIGO	Runner preparado para comprobar health, ready, auth, runtime, idempotencia, ciclo autónomo y evidencia.
Cadena de certificación	EXISTE EN CÓDIGO	Preflight, static gate, exact-SHA, evidence receipt, retained-evidence y políticas fail-closed.
Cockpit original MAP/DASHBOARD/CHAT	NO TERMINADO	No se ha materializado como aplicación final conectada a todos estos servicios.
VOICE del cockpit	NO TERMINADO	Forma parte de la experiencia final, pero no está cerrada como superficie operativa del producto.
Release certificado	NO CERTIFICADO	PR #18 sigue abierta, no mergeada y el estado declarado es NOT CERTIFIED.

Recorrido del desarrollo: qué se fue construyendo

La evolución técnica puede leerse como capas. Esto explica por qué hoy existe mucho “debajo” aunque todavía falte el cockpit que lo hace visible.

CAPA	TRABAJO REALIZADO
1 · Persistencia y modelo base	Creadores, proyectos, estado de proyecto, commitments, checkpoints, decisiones, knowledge, summaries, memoria, conflictos y solicitudes runtime.
2 · Concurrencia y continuidad	Versionado, leases, commits atómicos, idempotencia y fences para impedir escrituras o ejecuciones duplicadas.
3 · Efectos externos	Effect ledger, proveedor/authority binding, claims, verified completion, compensation y recuperación.
4 · Misiones autónomas	Mission policy/ledger, execution lock, attempt ledger, wakeups, resume triggers y recovery.
5 · Supervisión autónoma	Supervisor, health policy, detección de ejecuciones abandonadas, recovery scanner e incidentes.
6 · Human-in-the-loop	Human resolution, command inbox, incident claim/acknowledge/resolve y decisiones de next-stage.
7 · Governance	Registro de capacidades, cambios gobernados, aprobación/rechazo/aplicación y autoridad por dominio.
8 · Integridad / reparación	Lifecycle audit, integrity incidents, repair workflows, receipts, reconstruction y reconciliation.
9 · Producción / seguridad	JWT, secrets, readiness, schema invariant gate, bootstrap y database audit.
10 · Evidencia / certificación	Exact SHA, lockfile, PostgreSQL 18, zero-touch, evidence format, receipt verification y retained evidence.
11 · Presentación técnica	Launchers y runner de presentación que pueden generar evidencia SHA-bound del flujo técnico.
12 · Producto visible	PENDIENTE: convertir todo lo anterior en MAP + DASHBOARD + CHAT + BRAIN y cerrar VOICE.

Magnitud del hardening actual (PR #18, snapshot observado):

- 2.336 commits por delante de la base del PR.
- 492 archivos modificados dentro del PR.
- +39.886 / -1.677 líneas reportadas por GitHub para el PR.
- La magnitud del PR no equivale por sí sola a “producto terminado”; demuestra volumen de ingeniería, no aceptación de producto.

Qué prueba el código de demo que ya existe

El runner de presentación existente está diseñado para ejecutar un flujo técnico real, no una animación. En su código se verifica la siguiente secuencia:

PASO	PRUEBA PREVISTA
1	Crear aplicación de producción contra PostgreSQL 18.
2	Aplicar/verificar el schema esperado.
3	Arrancar Autonomous Runtime.
4	Comprobar /health.
5	Comprobar /ready: database + schema + AI + runtime.
6	Crear creator, project y project_state en PostgreSQL.
7	Generar JWT.
8	Comprobar que una llamada no autenticada retorna 401.
9	Ejecutar POST /v1/projects/{projectId}/runtime.
10	Repetir con la misma idempotency key y verificar misma respuesta.
11	Comprobar el registro durable en runtime_api_requests.
12	Ejecutar otro ciclo autónomo y verificar que termina sin error.
13	Apagar limpiamente.
14	Escribir evidencia JSON + receipt SHA-256 y verificarla.

Qué falta

Para considerar cumplido el objetivo mínimo de producto visible. Debe existir un circuito visual y comprobable.

FALTA	CONDICIÓN DE CIERRE
Cockpit frontend original	Construir una aplicación real con navegación MAP / DASHBOARD / CHAT / BRAIN. VOICE puede entrar después del primer alpha visible, pero no debe borrarse del alcance final.
Company Registry	Materializar una fuente canónica para Company, Department, Function y su relación con capabilities/agents/workflows. Es necesaria para que MAP sea dinámico y no un dibujo.
Read models para frontend	Endpoints/read models que entreguen al cockpit datos consolidados de company graph, missions, agents/capabilities, incidents, approvals, artifacts y activity.
MAP conectado	Cada nodo debe provenir de datos reales y permitir bajar de empresa → departamento → función → capacidad → ejecución.
Dashboard operacional	Mostrar health/readiness, misiones, actividad, incidentes, aprobaciones, bloqueos, runtime y resultados reales.
Chat operativo	Enviar una orden o consulta al runtime y devolver el resultado con estado/evidencia.
Persistencia visible	Ejecutar una acción, refrescar la aplicación y comprobar que el resultado sigue presente porque vive en PostgreSQL.
Auth usable	Login/sesión de demostración que produzca JWT/scopes válidos sin pegar tokens manualmente.
E2E de una historia concreta	Ejemplo: Dirección lanza misión → runtime procesa → estado cambia → si requiere autoridad aparece aprobación → se resuelve → resultado/evidencia aparece en dashboard.
Empaquetado de demo	Un solo comando o launcher que levante DB + backend + frontend y abra la URL de la demo.
Evidencia del demo	Captura/log/receipt del flujo y una receta reproducible para repetirlo.
Cierre de release	Congelar SHA, ejecutar gates, almacenar evidencia y sólo entonces considerar certificación/merge.

Cuánto trabajo falta para ver algo funcionar

Estimación de ingeniería enfocada, suponiendo que el backend actual arranca contra una instancia PostgreSQL 18 sin aparecer un bloqueo nuevo de integración. Son horas de trabajo efectivo, no fechas calendario garantizadas.

VENTANA	HITO	RESULTADO
0–6 h	Primera pantalla fiel	Shell visual del producto original, navegación MAP/DASHBOARD/CHAT, conexión a health/ready y estructura base. Se puede “ver”, pero todavía no es una demo E2E suficiente.
6–16 h	Conexión operativa mínima	Auth de demo, runtime de proyecto, incidents/capabilities read models y persistencia visible. Primeras acciones reales desde interfaz.
16–24 h	DEMO FUNCIONAL MÍNIMA	Un flujo completo desde dashboard → backend → PostgreSQL → resultado/evidencia → refresco. Este es el primer punto en que el fundador debería poder decir “sí, esto funciona”.
24–40 h	Alpha fiel al concepto	MAP dinámico con Company Registry inicial, departamentos/funciones/capabilities, dashboard operativo, chat integrado y estados reales.
40–60 h	Demo para terceros	Pulido, errores, loading, permisos, datos de demo coherentes, launcher único, pruebas y evidencia reproducible. VOICE puede cerrarse aquí o como siguiente bloque según estabilidad.
Después	Certificación del release	Congelar SHA, resolver discrepancias de naming/schema si las hubiera, ejecutar gates completos, retener evidencia y mergear según política.

HITO QUE IMPORTA AHORA

No abrir otra línea de documentación como prioridad. El siguiente gate de producto debe ser la DEMO FUNCIONAL MÍNIMA: abrir una URL, ejecutar una acción real, obtener resultado, refrescar y comprobar persistencia/evidencia.

HOY SE PUEDE MOSTRAR	TODAVÍA NO SE DEBE VENDER COMO HECHO
Arquitectura de producto definida y documentada.	Aplicación final Autonomous Company terminada.
Repositorio real con runtime, DB, governance, recovery, incidents y seguridad.	Cockpit original completo y operativo.
PR #18 con trabajo de hardening material y amplia batería de pruebas.	Release certificado / production-ready.
Catálogo de schema hasta versión lógica 71.	Company Map dinámico validado en producción.
Runner de demo técnico codificado con health/ready/auth/runtime/idempotency/evidence.	Prueba E2E canónica del HEAD actual.
Documentación de arquitectura, due diligence y reconstrucción.	VOICE totalmente integrado al Command Center.
Ruta concreta y corta hacia una demo visible.	Operación multi-área autónoma demostrada ante terceros.

“Autonomous Company ya tiene un motor técnico sustancial: runtime persistente, misiones, recuperación, gobierno de capacidades, seguridad, PostgreSQL y una cadena de evidencia/certificación. Lo que estamos cerrando ahora es la capa de producto que lo hace visible y operable: el cockpit MAP/DASHBOARD/CHAT.

Inventario técnico verificable del trabajo actual

Archivos/áreas representativas que sustentan este estado (lista no exhaustiva):

ACTIVO	QUÉ DEMUESTRA
backend/start.js	Entry point de aplicación.
backend/bootstrap/production.bootstrap.js	Composición de DB, AI provider, auth, APIs, runtime autónomo, readiness y servidor.
backend/api/runtime-api.module.js	Superficie de runtime de proyecto.
backend/auth/jwt-hs256.provider.js	Autenticación JWT HS256.
backend/project-runtime/production-autonomous-runtime-graph.factory.js	Composición del grafo autónomo de producción.
backend/project-runtime/autonomous-project-loop.js	Loop autónomo de proyecto.
backend/project-runtime/mission-execution-orchestrator.js	Orquestación de misiones.
backend/project-runtime/effect-execution-ledger.service.js	Ledger durable de efectos/ejecuciones.
backend/project-runtime/direction-command-api-boundary.js	Comandos de Dirección, incidentes, reparaciones y resolución.
backend/project-runtime/capability-governance-command-api.js	Governance de capacidades.
backend/project-runtime/initial-capability-business-api.js	Artifacts empresariales iniciales.
backend/database/migrations/migration.catalog.js	Autoridad del catálogo de migraciones; schema lógico actual 71.
database/migrations/001...069	Evolución durable del modelo y hardening.
tools/run-autonomous-company-presentation-demo.mjs	Demo técnica con evidencia.
tools/run-autonomous-company-presentation-demo-docker.mjs	Preparación/ejecución de demo con Docker.
RUN/PREPARE/CERTIFY-AUTONOMY-COMPANY-PRESENTATION.cmd	Launchers Windows.
docs/AUTONOMOUS_CERTIFICATION_*	Contrato, aceptación, autoridad, evidencia, failure taxonomy y handoff.
tests/* + tests/integration/*	Batería de pruebas unitarias/integración y gates de certificación.

Estado de control

ÁREA / COMPONENTE	ESTADO HOY	QUÉ SIGNIFICA
Repositorio	EXISTE	siandeda2-beep/creator-ai-os.
Rama observada	EXISTE	hardening/autonomous-v62-certification.
HEAD observado	EXISTE	287d68bf624f89a9ec8a645a56aa0dc3e31151b8.
PR #18	PENDIENTE	Open, mergeable, not merged; 2.336 commits sobre la base observada.
Schema lógico	IMPLEMENTADO	EXPECTED_SCHEMA_VERSION = 71.
Workflow observado	PENDIENTE	Run 35701244409: failure; runtime-validation sin steps expuestos, resto skipped.
Certificación	NO CERTIFICADO	El propio PR declara NOT CERTIFIED.
Dashboard original	NO TERMINADO	No se ha cerrado la experiencia MAP/DASHBOARD/CHAT/BRAIN como aplicación operativa.
Demo visible al fundador	NO ENTREGADA	No existe todavía una demostración de producto E2E aceptada por el fundador.

Plan de cierre

PASO	TRABAJO	SALIDA
A	Congelar el objetivo de la demo	MAP + DASHBOARD + CHAT + BRAIN visual; VOICE queda explícitamente como siguiente bloque si amenaza el primer E2E.
B	Crear Company Registry mínimo real	Empresa, departamentos, funciones y enlaces a capabilities/workflows. Sin esto el MAP sería sólo decoración.
C	Exponer read models	Company map, runtime status, missions, capabilities, incidents, approvals, artifacts, recent activity.
D	Construir frontend fiel	Mismo lenguaje visual/conceptual que la referencia original. Nada de consola genérica como producto final.
E	Conectar una historia E2E	Una orden desde UI atraviesa auth, runtime, persistencia, governance si aplica, y vuelve con resultado/evidencia.
F	Demostrar persistencia	Refresh/restart controlado sin perder el estado de la historia.
G	Empaquetar launcher	Start local reproducible de DB + backend + frontend.
H	Grabar evidencia	Log, receipt, SHA y captura del flujo.
I	Recién después: hardening de demo	Errores, UX, responsive, datos, retries, permisos.
J	Congelar/certificar release	Exact SHA + gates + retained evidence + merge bajo criterio de aceptación.

. Antes de una reunión crítica o certificación, actualizar esta página con un nuevo snapshot.